

Models Rmd

2026-06-24

Models

- Package Installation

```
library("rethinking")
library(dplyr)
library(dagitty)
```

Causal effects of the DAG

```
kdag<-dagitty("dag {
  S-> K <- T
  T -> S
  T -> SL -> K
}")
adjustmentSets(kdag, exposure="SL", outcome = "K")# This shows the backdoor path and how to stratify
## { T }
```

Synthetic Data

- create a synthetic data with assumptions. See overview.pdf to check the data relationship.

```
# Function of SL by Y
# This is arbitrary made by me, immitating the research by Susumu Tanabe and Rei Nakashima in 2026
# Y is between 0 and 1, which represents 16,000 years ago to 2,400 years ago.
myFunc <- function(x) {
  case_when(
    0 <= x & x <= (-9+ 16)/(13.6) ~ -(x * (-13.6)) * 60 / 7 - 80,
    (-9.000+ 16)/(13.6) < x & x <= (-7.500 + 16)/(13.6) ~ -(x * (-13.6) + 7) * 46 / 3 - 20,
    (-7.500+ 16)/(13.6) < x & x <= (-4.3+ 16)/(13.6) ~ 3,
    (-4.3 + 16)/(13.6) < x & x <= (-3.000+ 16)/(13.6) ~ -(x * (-13.6) + 11.7) * (-60) / 13 + 3,
    (-3.000+ 16)/(13.6) < x & x <= (-2.400+ 16)/(13.6) ~ -(x * (-13.6) + 13) * 5 / 3 - 1,
    TRUE ~ 1
  )
}
curve(myFunc(x), col = 2, main = "Relative Sea Level and Year")

# Initialize Y
Y<- runif(200, 0, 1)

# T and SL caused by Y
T<- NULL
SL<-NULL
for (i in 1:length(Y)) {
```

```

T[i]<-rpois(1, Y[i]* 25)
if(T[i] < 1) {T[i]<- 1}
else if(T[i] > 25) {T[i]<- 25} # Make sure T is between 1 and 25
SL[i]<- rnorm(1, myFunc(Y[i]), 5)
}

SL <- (SL - min(SL)) / (max(SL) - min(SL)) # to estimate easily, make it between 0 and 1

# Check the data
plot(T, SL, main = "Synthetic Data Plot(T and SL)")
plot(Y, T, main = "Synthetic Data Plot(Y and T)")
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")

# initialize dS with random numbers
dS<-rbeta(length(T), 1.2,1) * 200
# S is affected by Y and dS (omit T as T and S are correlated by Y)
S<-NULL
for(i in 1:length(T))S[i] <- rnorm(1, 5*Y[i] -dS[i]/50, 0.1)/25 + 0.3

# Check the data
range(S) # try to make less than 0.5

## [1] 0.1419801 0.4868649

plot(density(S), main = "Density of Synthetic Data of S")

plot(dS, S, main = "Synthetic Data Plot(dS and S) \n with different groups")
legend(x = 0, y = 0.18, legend="red: T = 2 blue: T = 7 green: T = 13")
points(dS[T==2], S[T==2], col = 2, lwd =4)
points(dS[T==13], S[T==13], col = 3, lwd =4)
points(dS[T==7], S[T==7], col = 4, lwd =4)
plot(T, S, main = "Synthetic Data Plot(T and S)")
plot(Y, S, main = "Synthetic Data Plot(Y and S)")
plot(T, dS, main = "Synthetic Data Plot(T and dS)\n No correlation expected.") # no correlation
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")

# Initialize dK with random numbers
dK<-rbeta(length(T), 1.4,1)*400
# K is affected by SL, Y, S, dK
K<-NULL
for(i in 1:length(T))K[i] <- rnorm(1, 1 * SL[i] + 1 * Y[i] + 1 * S[i]- dK[i]/250, 0.2)/10 + 0.2

# Check the data
range(K)# try to make less than 0.5

## [1] 0.07457748 0.44126745

plot(density(K), main = "Density of Synthetic Data of K")
plot(dK, K, main = "Synthetic Data Plot(dK and K)")
plot(SL, K, main = "Synthetic Data Plot(SL and K)")
plot(Y, K, main = "Synthetic Data Plot(Y and K)")
plot(S, K, main = "Synthetic Data Plot(S and K)")
plot(dS, K, main = "Synthetic Data Plot(dS and K)\n No correlation expected.") # no correlation

# Standardize data without SL, S, K (those are ratio)

```

```
d<-list(T=T, dS=standardize(dS), S=S, K=K, dK=standardize(dK))
```

```
# check the data
```

```
plot(density(d$K), main = "Density of Synthetic Data of d$K")
```

```
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
```

```
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
```

```
plot(Y, d$K, main = "Synthetic Data Plot(Y and d$K)")
```

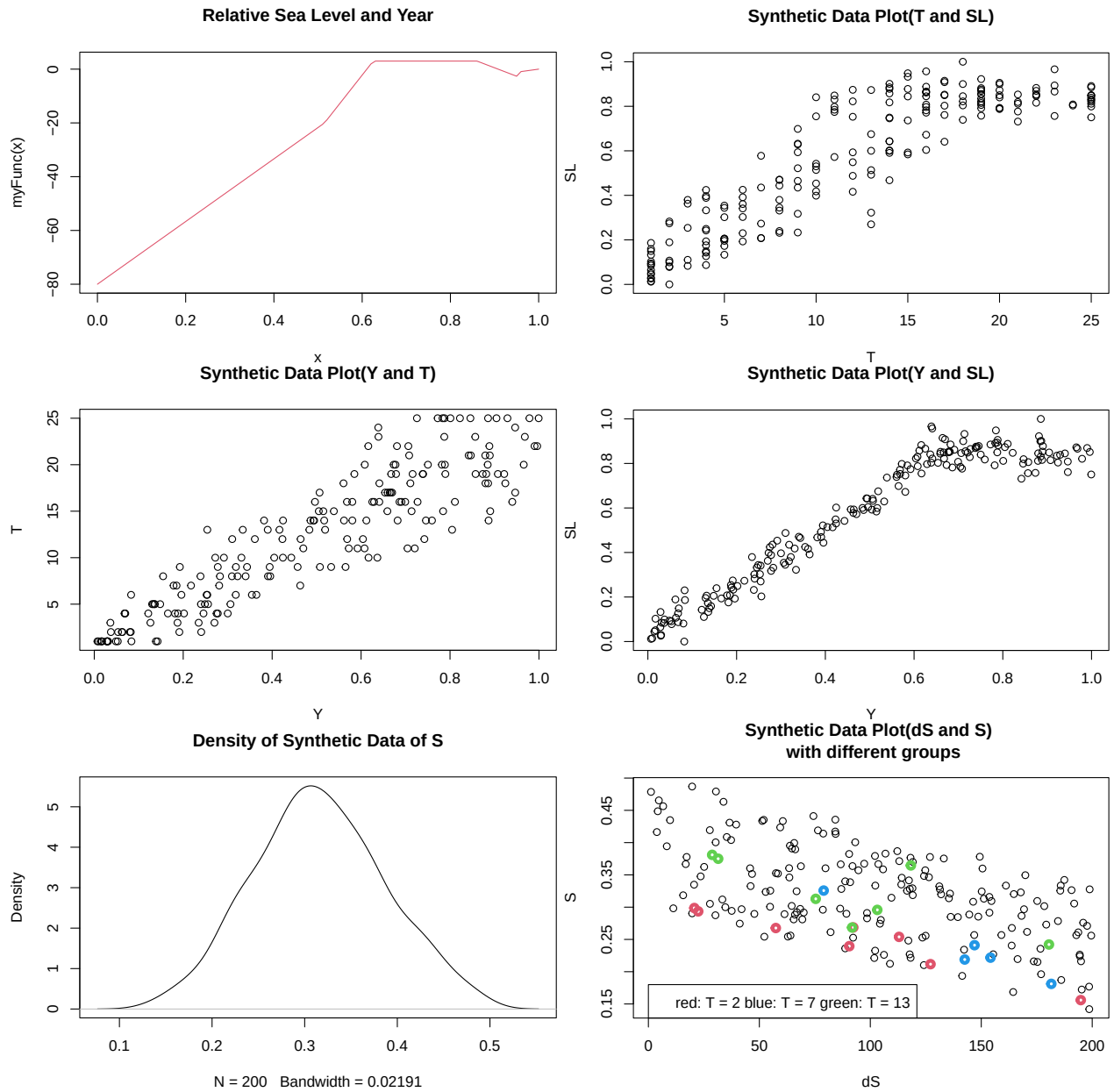
```
plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
```

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
```

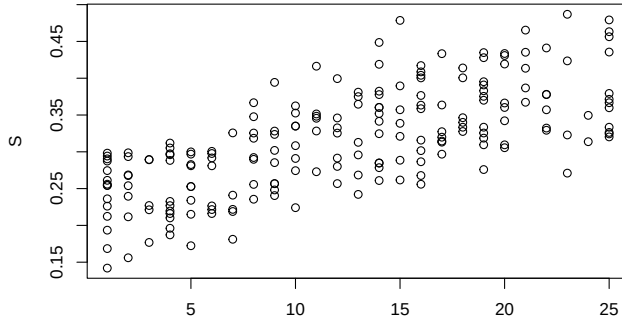
```
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
```

```
plot(Y, d$S, main = "Synthetic Data Plot(Y and d$S)")
```

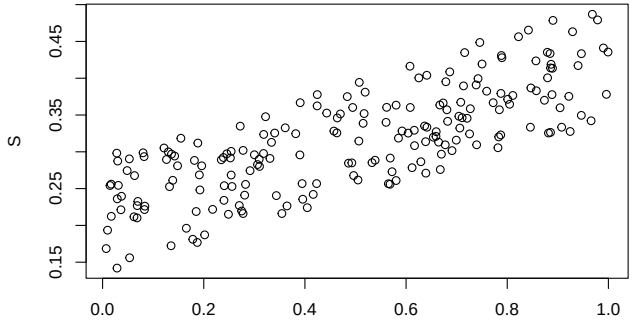
```
plot(d$dS, d$K, main = "Synthetic Data Plot(d$dS and d$K)\n No correlation expected.") # no correlation
```



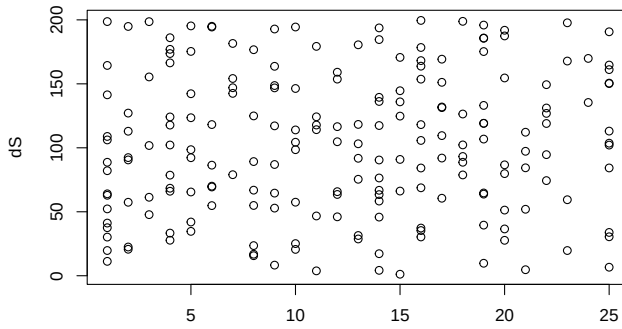
Synthetic Data Plot(T and S)



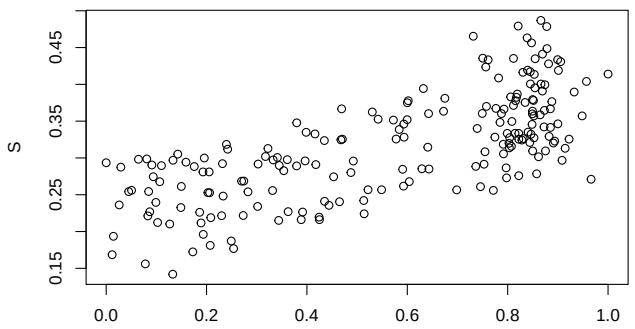
Synthetic Data Plot(Y and S)



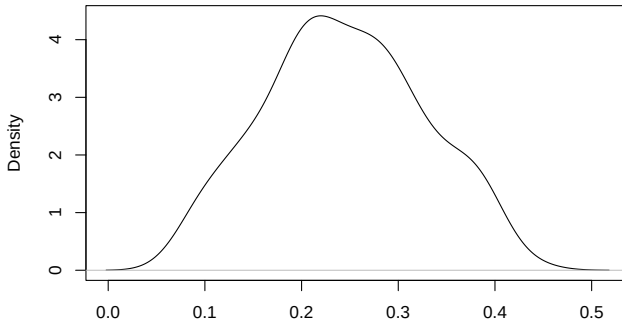
Synthetic Data Plot(T and dS)
No correlation expected.



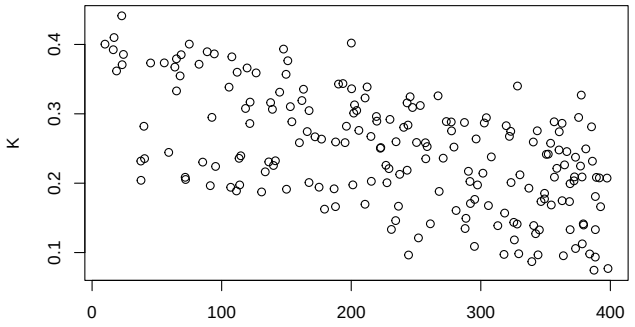
Synthetic Data Plot(SL and S)
auto-correlation



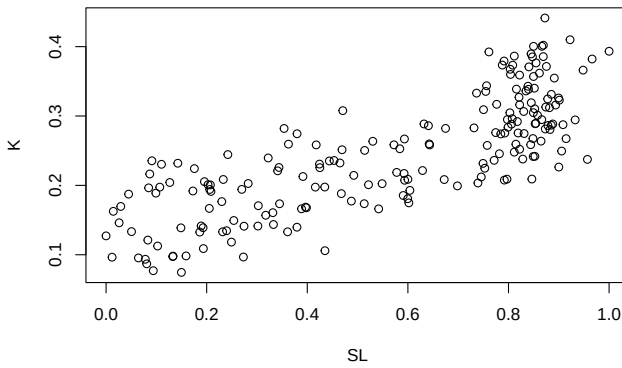
Density of Synthetic Data of K



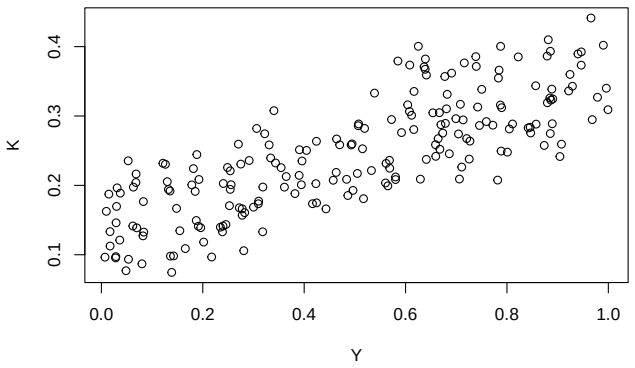
Synthetic Data Plot(dK and K)



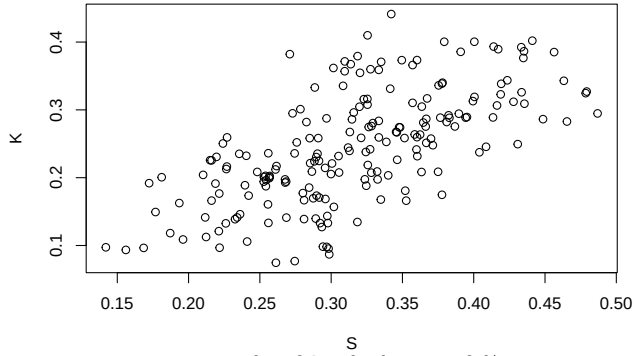
N = 200 Bandwidth = 0.02559
Synthetic Data Plot(SL and K)



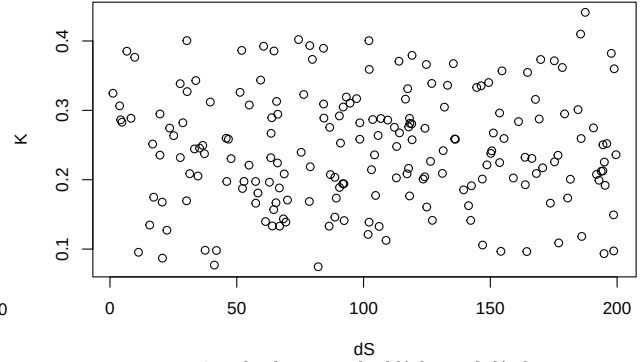
Synthetic Data Plot(Y and K)



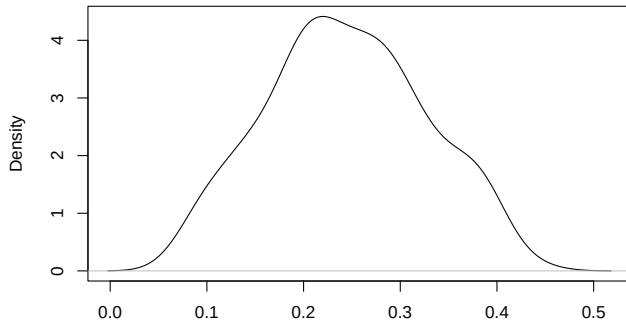
Synthetic Data Plot(S and K)



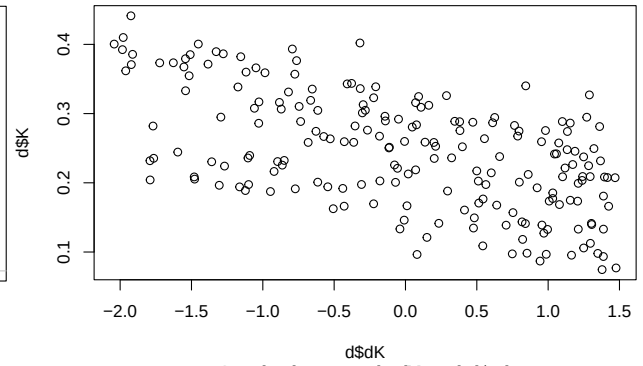
Synthetic Data Plot(dS and K)
No correlation expected.



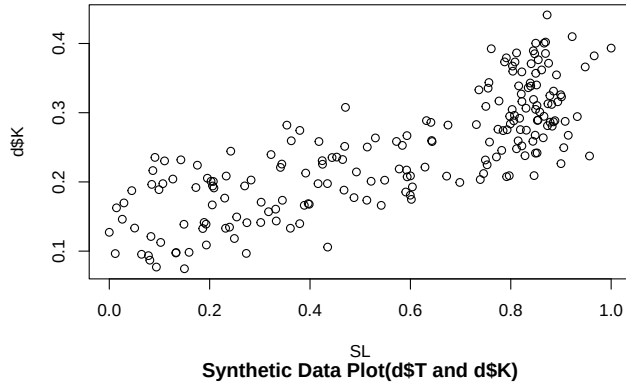
Density of Synthetic Data of d\$K



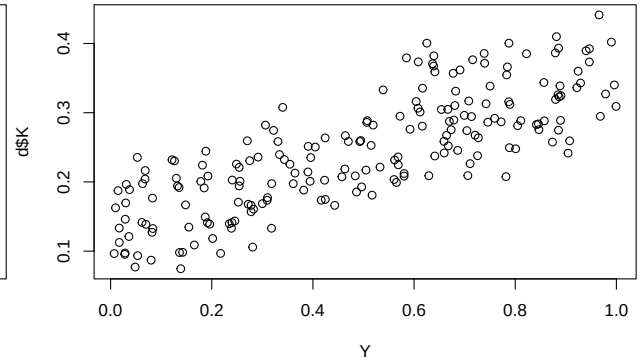
Synthetic Data Plot(d\$K and d\$K)



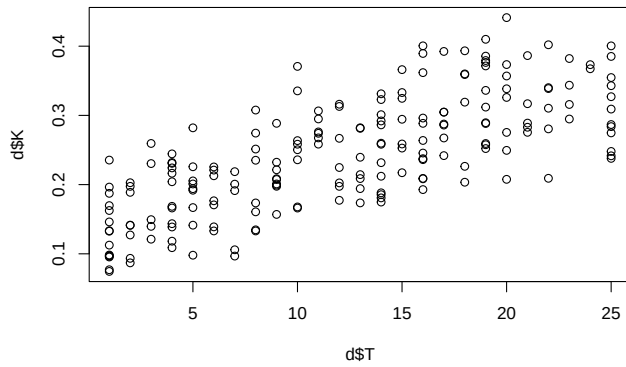
Synthetic Data Plot(SL and d\$K)



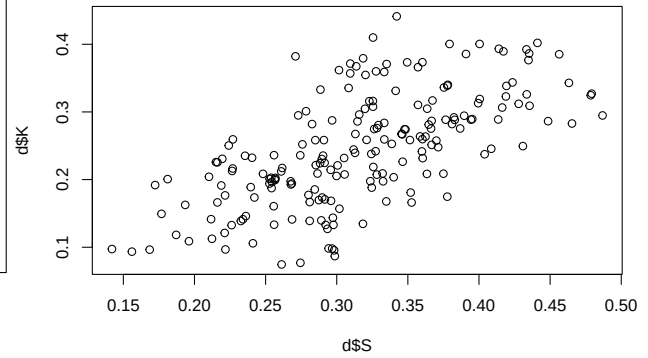
Synthetic Data Plot(Y and d\$K)

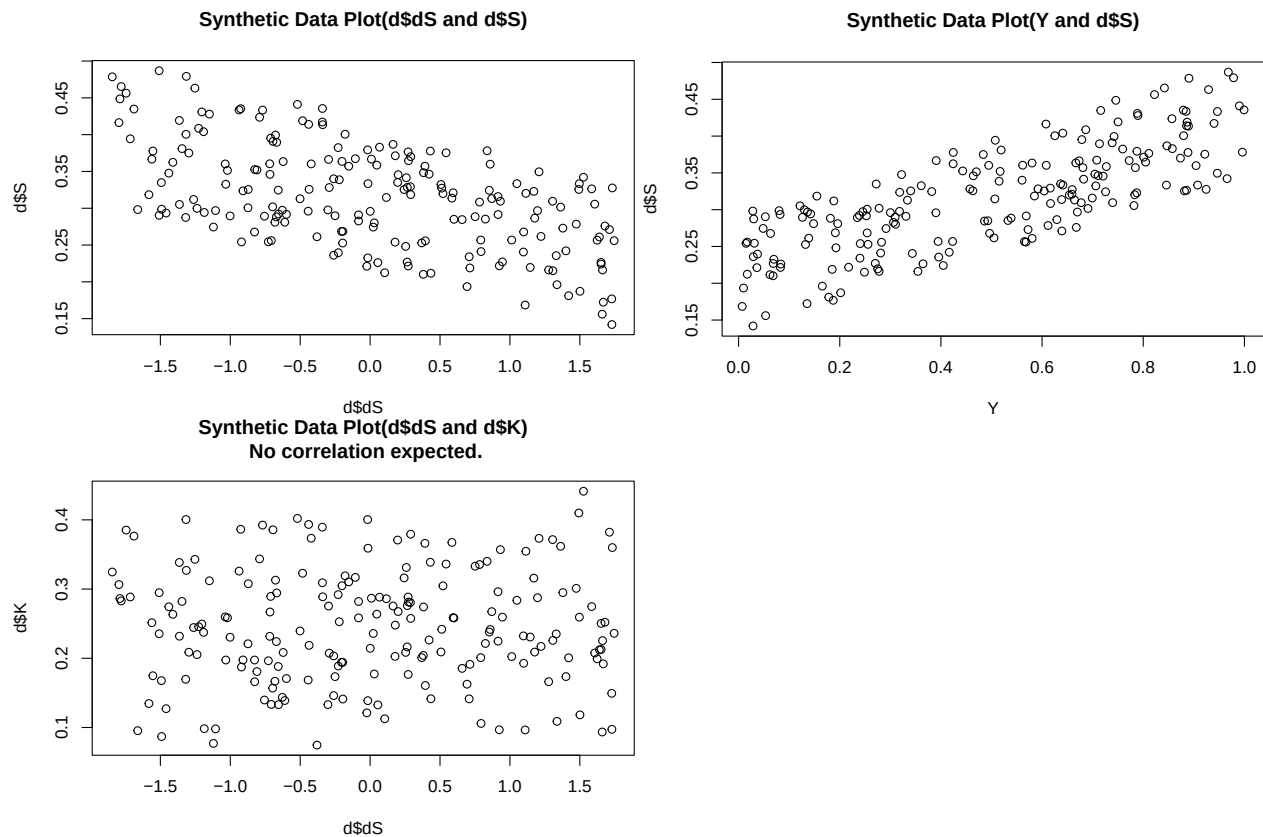


Synthetic Data Plot(d\$T and d\$K)



Synthetic Data Plot(d\$S and d\$K)

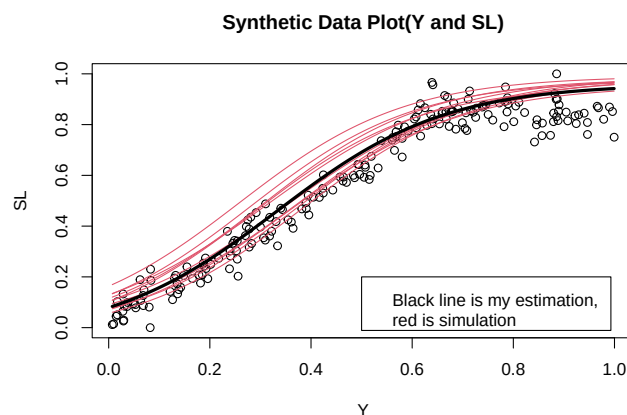




Prior Prediction

- Now I want to create a prior and statistical model.
- As in the overview, the Y and SL are hidden parameter so how to initialize is important
- This time, to make it simple, I use logit function to immitate the *myFunction*

```
plot(Y, SL, main = "Synthetic Data Plot(Y and SL)")
# Simulation
for(i in 1:10)curve(inv_logit(rnorm(1, 6, 0.2)*x + rnorm(1, -2, 0.2)) - rnorm(1, 0.04, 0.02), add =TRUE)
curve(inv_logit(6*x - 2) - 0.04, add =TRUE, lwd = 3)
legend(0.5, 0.2, legend="Black line is my estimation, \nred is simulation")
```



Model 1

- Same thing! See Overview pdf for details.

```
m1<- ulam(  
  alist (  
    # Main model K  
    K ~ normal(mu_K, U),  
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_Y * Y[i] + bK_dK * dK + bK_T[T],  
    # We do not know anything about this but think similar for all group.  
    # Mean should start at 0 and has small variation  
    # bK_T is regularized to maintain the efficiency.  
  
    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0,0.1), # This is different from the other one, so th  
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],  
    bK_T_bar ~ normal(0, 0.1), # regularization but not high sigma  
    sigma_bK_T ~ exponential(1), # Needs to be 1 to maintain bK_T.  
    z_bK_T[T] ~ normal(0, 0.1),  
  
    U ~ exponential(1),  
  
    # Model S  
    S ~ normal(mu_S, sigma_S),  
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],  
  
    # I am hoping/believing b_dS is negative, b_Y_S is positive, and bS_T is different for each one  
    c(aS, bS_Y, bS_dS) ~ normal(0,0.1),  
    # bS_T is regularized.  
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],  
    bS_T_bar ~ normal(0, 0.1),  
    sigma_bS_T ~ exponential(1),  
    z_bS_T[T] ~ normal(0, 0.1),  
  
    sigma_S ~ exponential(1),  
  
    # Model SL by Y  
    # The log function and prior was refered to the previous research of Sea Level Change over time  
    # This function is abstract so I will have to actually stan code to provide accurate estimates  
    # ulam code in r does not allow us to hard code the conditional function.  
    # vector[200]: SL ~ normal(muSL + aSL2, sigmaSL),  
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),  
  
    # Model T, This should be hard coded based on the previous research  
    T ~ poisson(Y * 25),  
  
    # Model T by Y  
    vector[200]:Y ~ normal(0.5, 0.3)  
  
  ), dat = d, constraints = list(  
    SL = "lower=0,upper=1", # SL is between 0 and 1  
    Y = "lower=0,upper=1" # Y is between 0 and 1  
    #, T = "lower=1,upper=25" #this is unnecessary since T is provided in the dat  
  ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(  
    adapt_delta = 0.99  
  )  
)
```

```

)

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
##     | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f307cd01c9.hpp:1308:3: note: by 'void ulam_cmds
## 1308 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##     | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##     | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f307cd01c9.hpp:1308:3: note: by 'void ulam_cmds
## 1308 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##     | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
##     | ~~~~~
## C:/Users/n
##
## obut/AppData/Local/Temp/RtmpM7wHH9/model-f307cd01c9.hpp:1345:3: note: by 'T_ ulam_cmdstanr_0e3bb37
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##     | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##     | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f307cd01c9.hpp:1345:3: note: by 'T_ ulam_cmdstan
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##     | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log
## 91 | inline double log_prob(Eigen::VectorXd& theta,
##     | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f307cd01c9.hpp:1345:3: note: by 'T_ ulam_cmdstan
## 13
## 45 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##     | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_cr
## 159 | inline math::var log_prob(std::vector<math::var>& theta,
##     | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f307cd01c9.hpp:1345:3: note: by 'T_ ulam_cmdstan
## 1345 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##     | ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like cova
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1

```



```

## Chain 2 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like cova
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 3 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like cova
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 4 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:    100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:    200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:    300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:    400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:    600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:    500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:    700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:    800 / 4000 [20%] (Warmup)

```

```

## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)

```

```

## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)

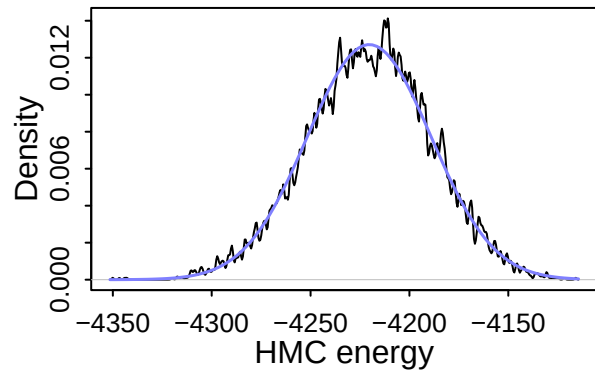
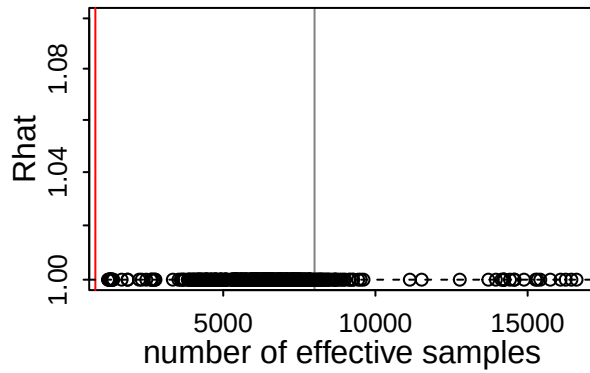
```

```

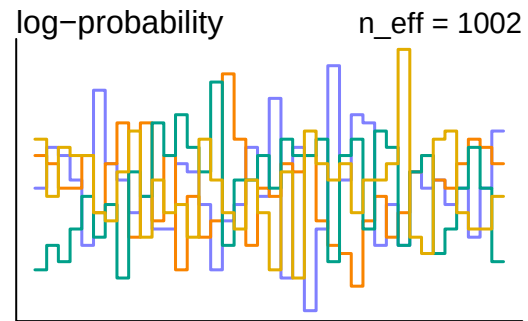
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 148.3 seconds.
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 175.1 seconds.
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 177.1 seconds.
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 187.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 172.0 seconds.
## Total execution time: 187.5 seconds.

```

```
dashboard(m1)
```



0
Divergent transitions
Outlook good



- without divergent but it has a few problem with efficiency. I will try to eliminate by removing parameters.

```
m1.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

    c(aK, bK_S, bK_SL, bK_Y, bK_dK) ~ normal(0,0.1),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1),
    sigma_bK_T ~ exponential(1),
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_Y * Y[i] + bS_T[T],

    c(aS, bS_Y, bS_dS) ~ normal(0,0.1),
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
```

```

# Model T
  T ~ poisson(Y * 25),

# Model T by Y
  vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL   = "lower=0,upper=1",
  Y    = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 1 Iteration:   900 / 4000 [22%] (Warmup)

```

```

## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)

```

[illegible]


```

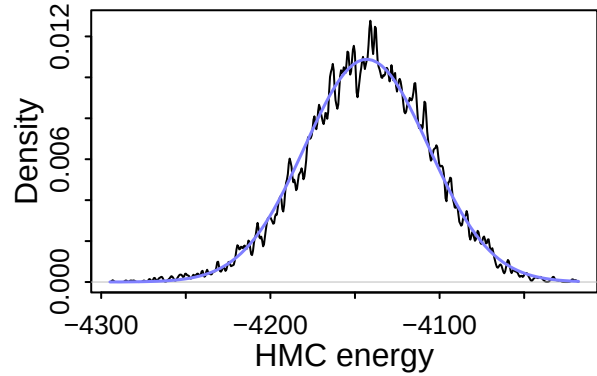
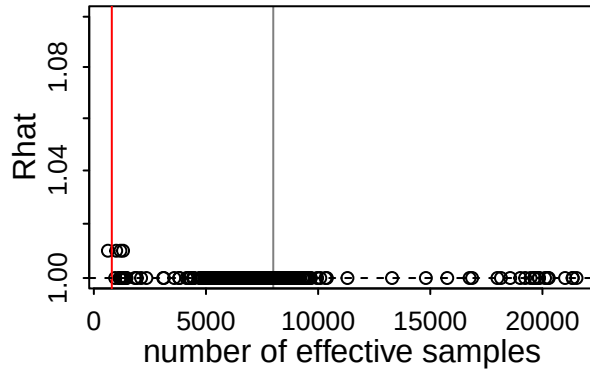
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 174.4 seconds.
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 179.5 seconds.
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 180.4 seconds.
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 185.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 180.0 seconds.
## Total execution time: 186.0 seconds.

```

```

dashboard(m1.2)

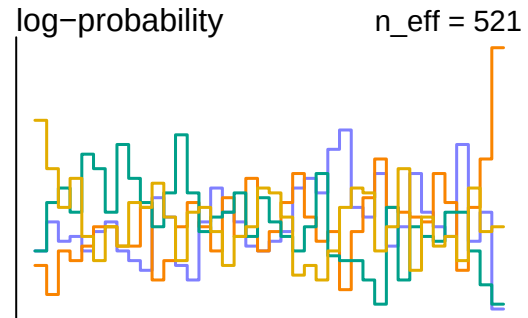
```



0

Divergent transitions

Outlook good



- This is still not the best model with low efficiency. I will remove one more parameter.
- I can remove bK_T or bS_T but this time, I want to make this similar to m1.3 and m2.3.

```
m1.3 <- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK + bK_S * S + bK_SL * SL[i] + bK_dK * dK + bK_T[T],

    c(aK, bK_S, bK_SL, bK_dK) ~ normal(0, 0.1),
    vector[25]: bK_T <- bK_T_bar + sigma_bK_T * z_bK_T[T],
    bK_T_bar ~ normal(0, 0.1),
    sigma_bK_T ~ exponential(1),
    z_bK_T[T] ~ normal(0, 0.1),

    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS + bS_dS * dS + bS_T[T],

    c(aS, bS_dS) ~ normal(0, 0.1),
    vector[25]: bS_T <- bS_T_bar + sigma_bS_T * z_bS_T[T],
    bS_T_bar ~ normal(0, 0.1),
    sigma_bS_T ~ exponential(1),
    z_bS_T[T] ~ normal(0, 0.1),

    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
```

```

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL  = "lower=0,upper=1",
  Y   = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   900 / 4000 [22%] (Warmup)

```

```

## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration:  700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration:  900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration:  800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration:  900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)

```

```

## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)

```

```

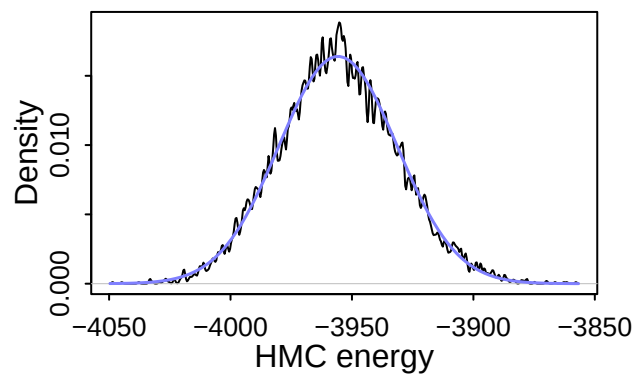
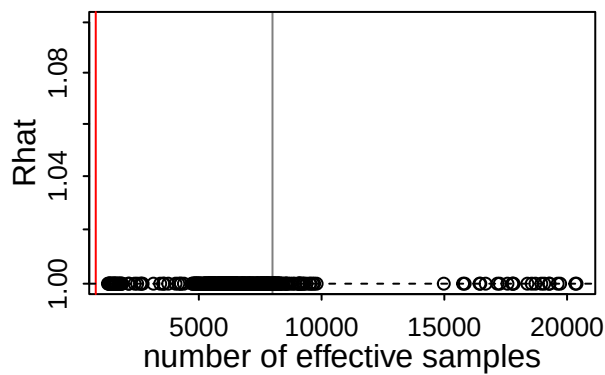
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 82.1 seconds.
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 83.6 seconds.
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 finished in 87.4 seconds.
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 113.2 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 91.6 seconds.
## Total execution time: 113.4 seconds.

```

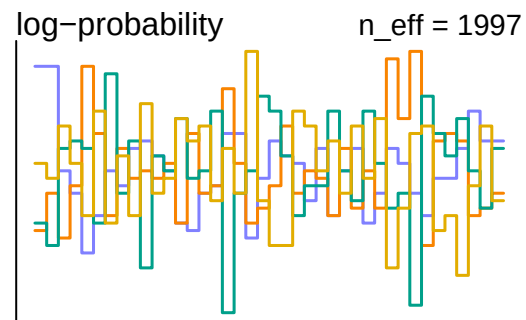
```

dashboard(m1.3)

```



0
Divergent transitions
Outlook good



```
precis(m1.3, depth = 1)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
## bK_dK	-0.22808426	0.014869260	-0.25200343	-0.20416430	1.0009259	15763.383
## bK_SL	0.35754914	0.095650239	0.20380518	0.50673228	1.0026377	2657.265
## bK_S	0.20683921	0.098770067	0.05050973	0.36477057	1.0001385	9121.018
## aK	-0.22267058	0.111219611	-0.40138122	-0.04314470	1.0006502	3425.943
## bK_T_bar	-0.22108727	0.109698604	-0.40181776	-0.05025466	1.0004333	3582.456
## sigma_bK_T	8.39587135	1.796075435	5.68743988	11.39664987	1.0018683	1546.295
## U	0.03772265	0.002591394	0.03375007	0.04199333	1.0015264	3150.087
## bS_dS	-0.21069427	0.011555077	-0.22912454	-0.19186432	1.0001324	17256.678
## aS	-0.29723189	0.085492394	-0.43123204	-0.15560194	0.9999624	4051.662
## bS_T_bar	-0.29626554	0.084636377	-0.42892158	-0.15911969	1.0010670	4267.123
## sigma_bS_T	2.91655449	0.733628912	1.99255553	4.25730438	1.0036610	1722.975
## sigma_S	0.03421450	0.001766116	0.03152334	0.03714164	1.0003601	14978.039

Posterior Check

```
post <- extract.samples(m1.3)
```

Hidden parameters

- The hidden parameters are estimated by the model constraints and distributions.
- It is expected that the difference is small and has similar trend as the original data.

```
# Posterior Y changes
plot(1:200, Y, main = "Difference between original (black) and simulated Y (red)")
points(1:200, post$Y[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$Y[1,i], Y[i]), col = 2)

plot(1:200, SL, main = "Difference between original (black) and simulated SL(red)")
```

```

points(1:200, post$SL[1,], col = 2)
for(i in 1 : 200) lines(c(i, i), c(post$SL[1,i], SL[i]), col = 2)

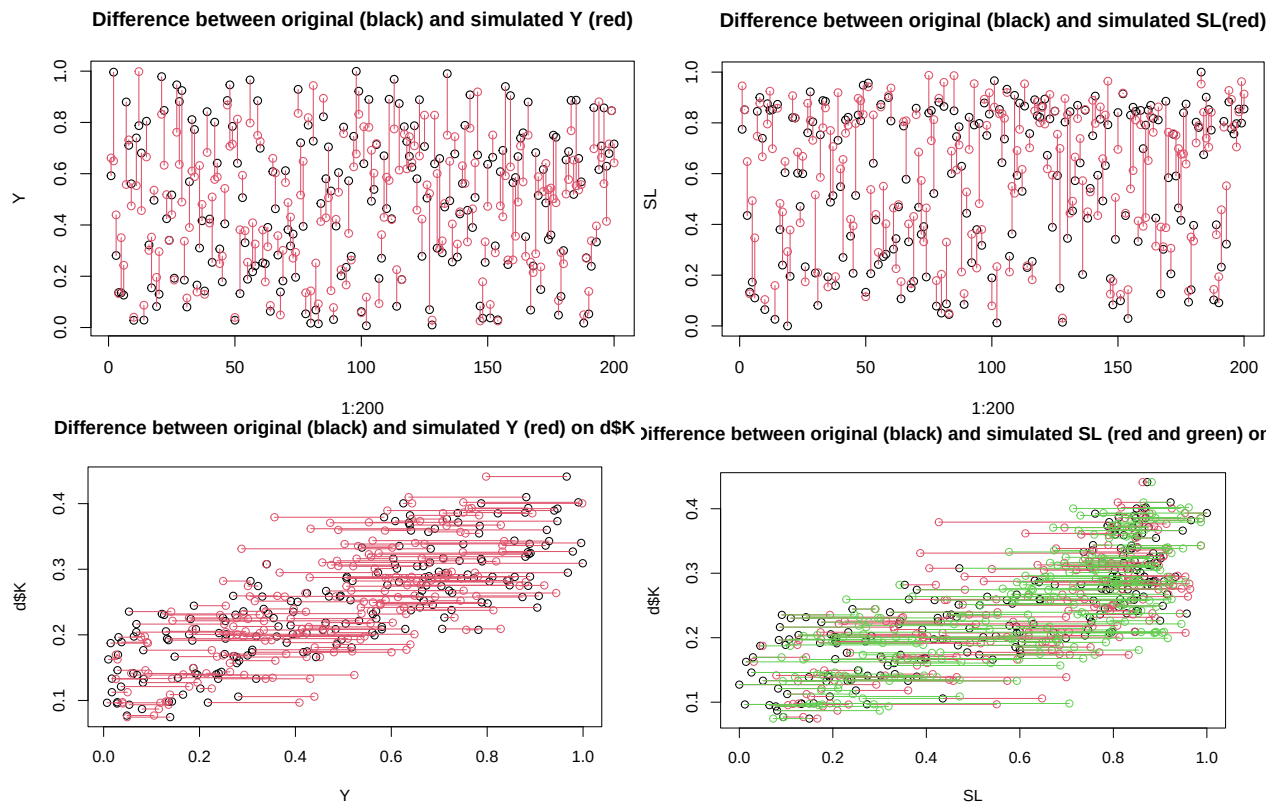
# check Y and SL with K
plot(Y, d$K, main = "Difference between original (black) and simulated Y (red) on d$K")
points(post$Y[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$Y[1,i], Y[i]), c(d$K[i], d$K[i]), col = 2)

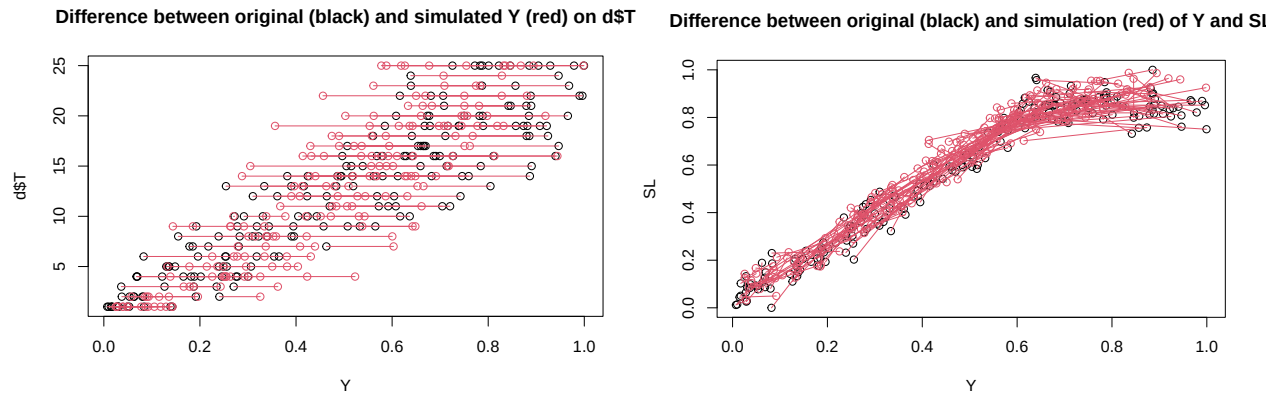
plot(SL, d$K, main = "Difference between original (black) and simulated SL (red and green) on d$K")
points(post$SL[1,], d$K, col = 2)
for(i in 1 : 200) lines(c(post$SL[1,i], SL[i]), c(d$K[i], d$K[i]), col = 2)
points(post$SL[2,], d$K, col = 3)
for(i in 1 : 200) lines(c(post$SL[2,i], SL[i]), c(d$K[i], d$K[i]), col = 3)

# Y and T
plot(Y, d$T, main = "Difference between original (black) and simulated Y (red) on d$T")
points(post$Y[1,], d$T, col = 2)
for (i in 1:200) lines(c(Y[i], post$Y[1,i]), c(d$T[i],d$T[i]), col = 2)

# Y and SL both hidden parameters
plot(Y, SL, main = "Difference between original (black) and simulation (red) of Y and SL")
points(post$Y[1,], post$SL[1,], col=2)
for(i in 1 : 200) lines(c(Y[i], post$Y[1,i]), c(SL[i], post$SL[1,i]), col = 2)

```





=> this shows that the difference is not very large and hidden parameters have similar trend as the original (synthesized) data.

Posterior Simulation

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq,
  function(i) inv_logit(post$aK + post$bK_S * i + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$dK)
)
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
points(post$SL[1,], d$K, col = 2) # include the estimated points, though we confirmed above that it is
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
  function(i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * i + post$bK_dK * d$dK + colMeans(post$SL))
)
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function(i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * i
)
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

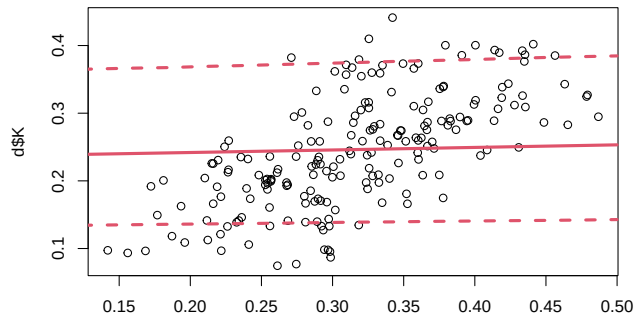
plot(d$T, d$K, main = "Synthetic Data Plot(d$T and d$K)")
KbyT<- sapply(1:25,
```

```

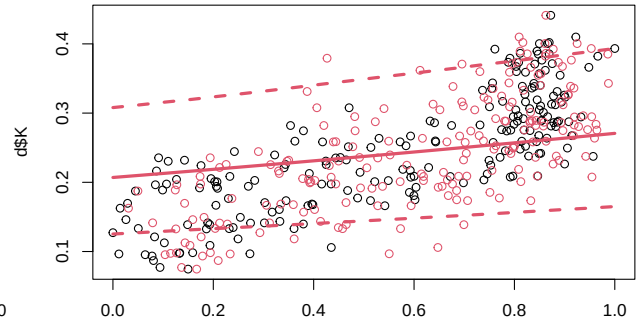
function (i) inv_logit(post$aK + post$bK_S * d$S + post$bK_SL * colMeans(post$SL) + post$bK_dK * d$
PIs<- apply( KbyT , 2 , PI )
means<- apply( KbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2 )
points(1:25, PIs[2, ], col = 2, lty = 2 )
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i])), col = 2)

```

Synthetic Data Plot(d\$S and d\$K)

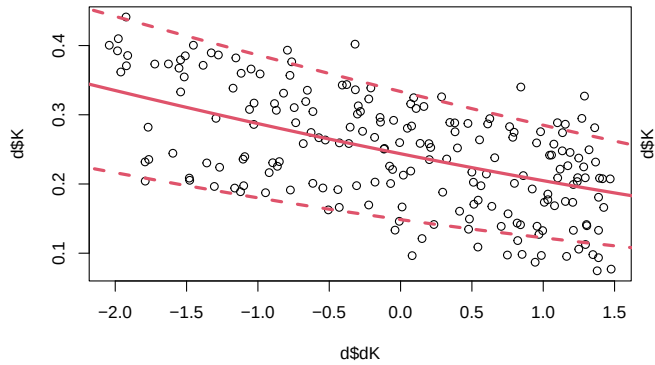


Synthetic Data Plot(SL and d\$K)

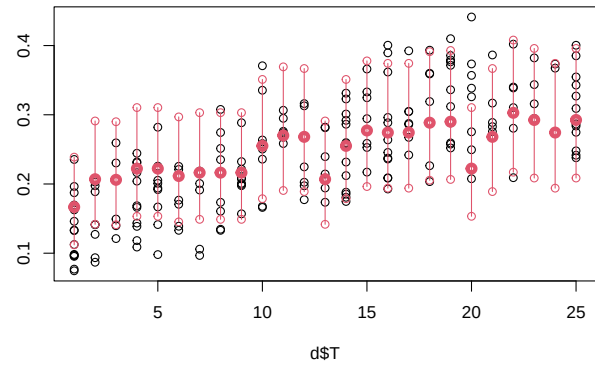


Model K

Synthetic Data Plot(d\$dK and d\$K)



Synthetic Data Plot(d\$T and d\$K)



- Looks nice!

Model S

- Do the same thing for S
- By the way, there is no post-S exists as it seems absolutely same as post-SL
- would be because it is one of the predictors that i provided in the data.

```
post$SL[1,] == post$S[1,]
```

```

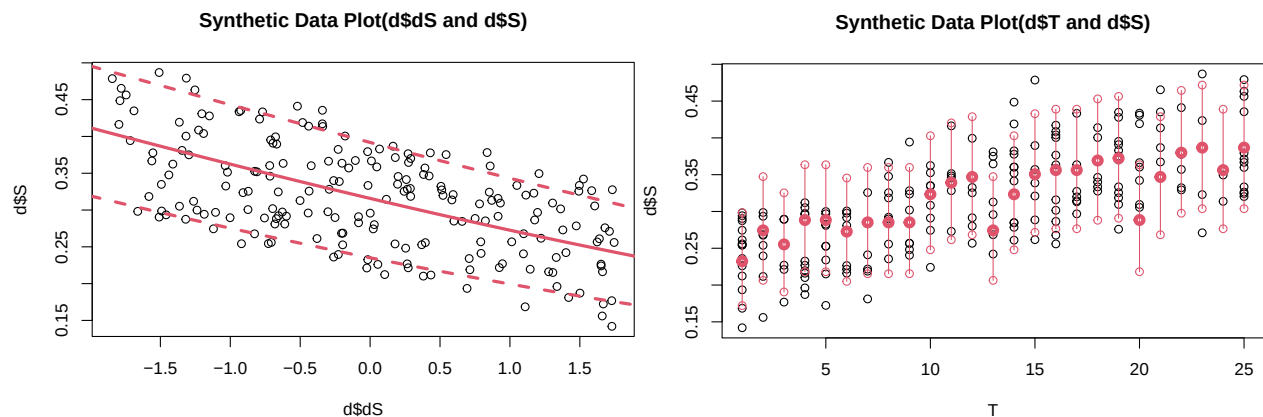
## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [16] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [31] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [46] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [76] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [91] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [106] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [136] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [151] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE

```

```
## [166] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [181] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
## [196] TRUE TRUE TRUE TRUE TRUE
```

```
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + post$bS_dS * i + colMeans(post$bS_T)[d$T] )) # Used mean of each s
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(T, d$S, main = "Synthetic Data Plot(d$T and d$S)")
SbyT<- sapply(1:25,
  function (i) inv_logit(post$aS + post$bS_dS * d$dS + colMeans(post$bS_T)[i]))
PIs<- apply( SbyT , 2 , PI )
means<- apply( SbyT , 2 , mean )
points(1:25, means, col = 2, lwd = 4)
points(1:25, PIs[1, ], col = 2, lty = 2)
points(1:25, PIs[2, ], col = 2, lty = 2)
for(i in 1:25) lines(c(i,i), c(PIs[1, i], PIs[2, i]), col = 2)
```



I have no idea why it does not model bK_T and bS_T very well... I will figure it out. Other than that, looks great.

Model 2

- Also see overview.pdf to know statistical model and prior
- This model stratifies all parameters by T.
- There are a few messages to warning my priors. Since it only shows up during first iteration of warmup, we can safely ignore it.
- I just omit it for m2.2 and 2.3.

```
# Model 2
m2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,
```

```

# Basically same as m1 but regularization (partial pooling).
# This does not need correlation because multilevel models of S and SL includes the information
# But we have to regularize
vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
c(aK_bar, bK_S_bar, bK_SL_bar, bK_Y_bar) ~ normal(0,0.3),
c(sigma_aK, sigma_bK_S, sigma_bK_SL, sigma_bK_Y) ~ exponential(6), # want low variation among g
z_aK[T] ~ normal(0,0.1),
z_bK_S[T] ~ normal(0,0.1),
z_bK_SL[T] ~ normal(0,0.1),
z_bK_Y[T] ~ normal(0,0.1),

# don't need regularization as dK is not affected by T
bK_dK ~ normal(0, 0.3),
U ~ exponential(1),

# Model S
S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS,

# Belows are similar to m1.
# partial pooling for the same reason for a2 and b_Y_S
vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
c(sigma_aS, sigmabS_Y) ~ exponential(6),
z_aS[T] ~ normal(0,0.1),
zbS_Y[T] ~ normal(0,0.1),

# don't need regularization
bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal(0.5, 0.3)

), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
## | ~~~~~

```



```

## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 4 Iteration:      1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of th
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like cova
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 3 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:   100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:   200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:   300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 1 Iteration:   400 / 4000 [10%] (Warmup)
## Chain 2 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 1 Iteration:   500 / 4000 [12%] (Warmup)
## Chain 3 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 4 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 1 Iteration:   600 / 4000 [15%] (Warmup)
## Chain 3 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 4 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 2 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 1 Iteration:   700 / 4000 [17%] (Warmup)
## Chain 3 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 4 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 2 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 1 Iteration:   800 / 4000 [20%] (Warmup)
## Chain 3 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 4 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 2 Iteration:  1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration:   900 / 4000 [22%] (Warmup)
## Chain 3 Iteration:  1000 / 4000 [25%] (Warmup)
## Chain 2 Iteration:  1100 / 4000 [27%] (Warmup)
## Chain 4 Iteration:  1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration:  1000 / 4000 [25%] (Warmup)
## Chain 2 Iteration:  1200 / 4000 [30%] (Warmup)

```

```

## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)

```

```

## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 finished in 43.7 seconds.
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)

```



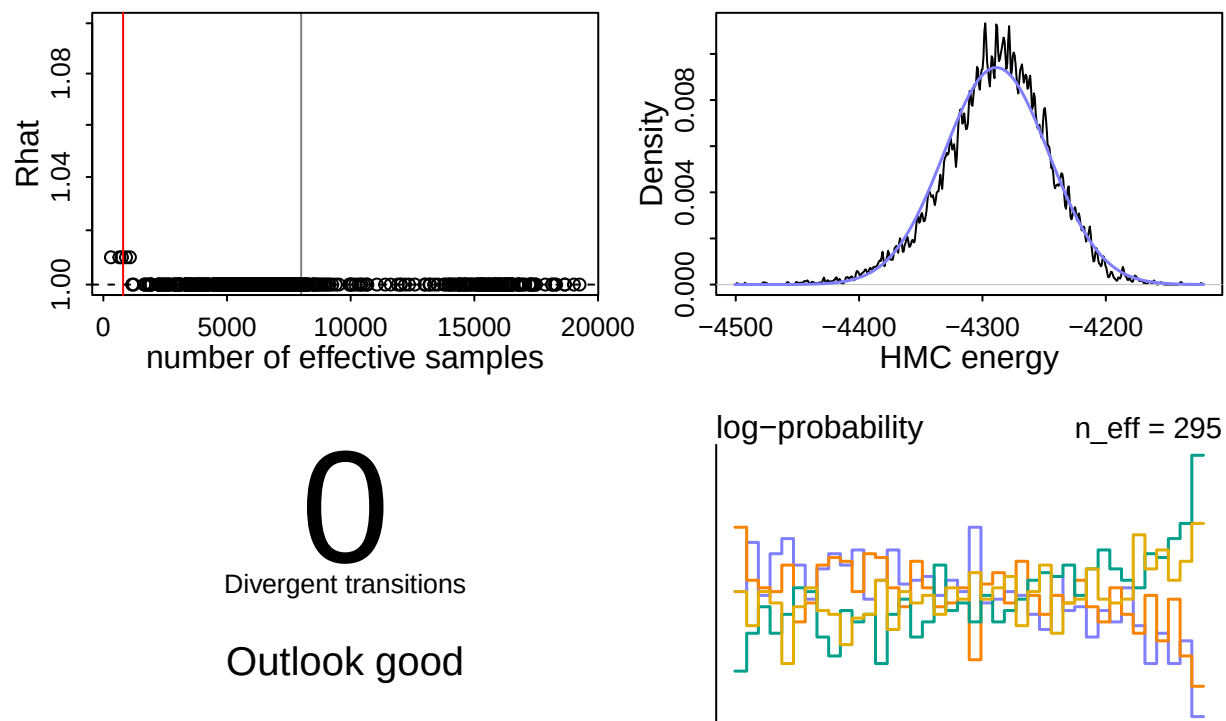
```

## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 51.8 seconds.
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 52.5 seconds.
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 53.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 50.3 seconds.
## Total execution time: 53.3 seconds.

## Warning: 1 of 4 chains had an E-BFMI less than 0.3.
## See https://mc-stan.org/misc/warnings for details.

```

dashboard(m2)



- This has so much divergent and less effective sampling.
- This would be due to the large number of parameters, including hidden estimated ones (SL and Y). I will reduce parameters to reliably interpret the result.
 - First one would be Y as S and SL are more important for the model.

* by the way, SL and Y in model K change the efficiency in the highest rate.

```
# Model 2.2
m2.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    c(aK_bar, bK_S_bar, bK_SL_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    S ~ normal(mu_S, sigma_S),
    logit(mu_S) <- aS[T] + bS_Y[T] * Y[i] + bS_dS * dS ,
    vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
    vector[25]: bS_Y<-bS_Y_bar + sigmabS_Y * zbS_Y[T],
    c(aS_bar, bS_Y_bar) ~ normal(0,0.3),
    c(sigma_aS, sigmabS_Y) ~ exponential(6),
    z_aS[T] ~ normal(0,0.1),
    zbS_Y[T] ~ normal(0,0.1),
    bS_dS ~ normal(0,0.3),
    sigma_S ~ exponential(1),

    # Model SL by Y
    vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

    # Model T, This should be hard coded based on the previous research
    T ~ poisson(Y * 25),

    # Model T by Y
    vector[200]:Y ~ normal( 0.5, 0.2)

  ), dat = d, constraints = list(
    SL = "lower=0,upper=1"
    , Y = "lower=0,upper=1"
  ), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)
```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
```

```

## Chain 4 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)

```

```

## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)

```

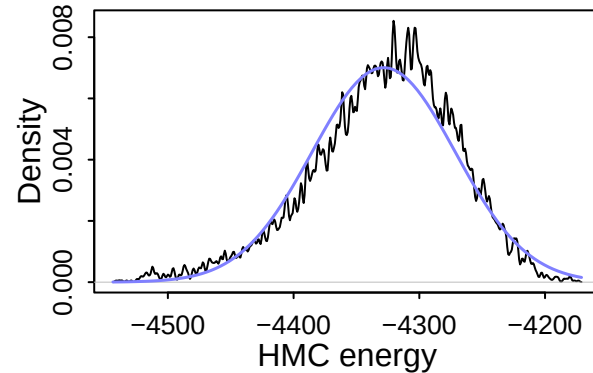
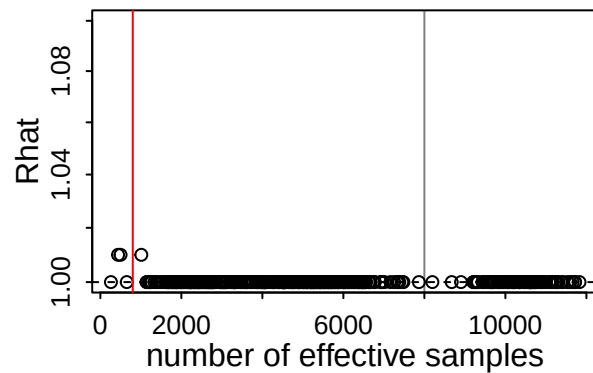
```

## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 36.0 seconds.
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 38.2 seconds.
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)

```

```
## Chain 3 finished in 42.5 seconds.
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 44.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 40.4 seconds.
## Total execution time: 45.0 seconds.
```

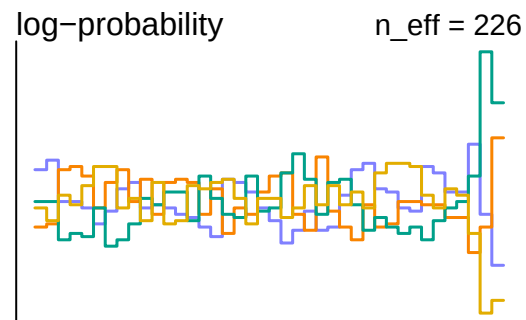
`dashboard(m2.2)`



7

Divergent transitions

Roll again



This is still bad model as the sampling efficiency is very small number

```
# Model 2.3
m2.3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,

    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_S <- bK_S_bar + sigma_bK_S * z_bK_S[T],
    vector [25]: bK_SL <- bK_SL_bar + sigma_bK_SL * z_bK_SL[T],
    c(aK_bar, bK_S_bar, bK_SL_bar) ~ normal(0,0.3),
    c(sigma_aK, sigma_bK_S, sigma_bK_SL) ~ exponential(6),
    z_aK[T] ~ normal(0,0.1),
    z_bK_S[T] ~ normal(0,0.1),
    z_bK_SL[T] ~ normal(0,0.1),
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),
```

```

# Model S
S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_dS * dS,
vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
aS_bar ~ normal(0,0.3),
sigma_aS ~ exponential(6),
z_aS[T] ~ normal(0,0.1),
bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),
# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)

), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE
)

```

Running MCMC with 4 parallel chains, with 1 thread(s) per chain...

```

##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 4 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 2 Iteration:  400 / 4000 [10%] (Warmup)
## Chain 3 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 1 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:  600 / 4000 [15%] (Warmup)
## Chain 2 Iteration:  500 / 4000 [12%] (Warmup)
## Chain 4 Iteration:  700 / 4000 [17%] (Warmup)

```

```

## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)

```



```

## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)

```

```

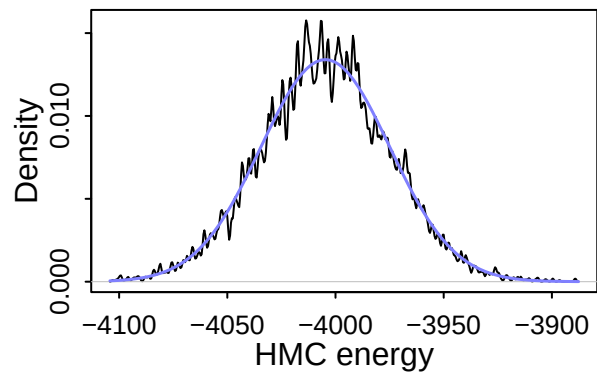
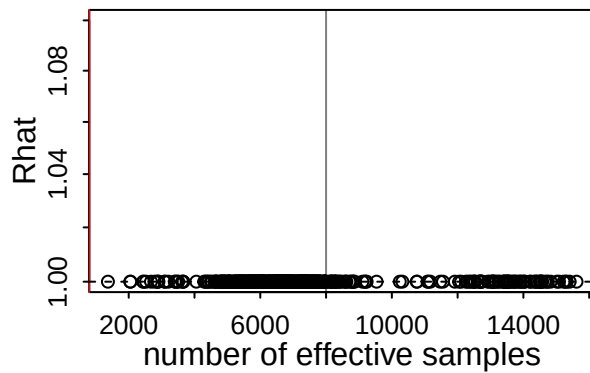
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 finished in 26.3 seconds.
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 26.8 seconds.
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 30.7 seconds.
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 32.8 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 29.1 seconds.
## Total execution time: 32.9 seconds.

```

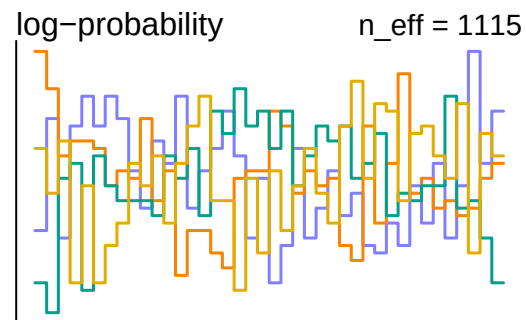
```

dashboard(m2.3)

```



0
Divergent transitions
Outlook good



```
precis(m2.3, depth=1)
```

	mean	sd	5.5%	94.5%	rhat	ess_bulk
## bK_SL_bar	1.15548472	0.064585242	1.052520731	1.25689192	1.000647	2056.041
## bK_S_bar	1.01712422	0.182002732	0.730865977	1.30913487	1.000386	4360.320
## aK_bar	-2.19903582	0.056769181	-2.287008135	-2.10670815	1.000818	2867.465
## sigma_bK_SL	0.12136884	0.111573259	0.008096024	0.33297450	1.000209	5094.264
## sigma_bK_S	0.15664076	0.151758335	0.009374393	0.45108012	1.000814	6732.982
## sigma_aK	0.13171252	0.119899804	0.008546768	0.36865237	1.000952	2785.548
## bK_dK	-0.23727920	0.011330752	-0.255146994	-0.21943494	1.000264	4824.368
## U	0.02038177	0.002203534	0.016918068	0.02397187	1.001013	1368.588
## aS_bar	-0.74703107	0.048754262	-0.823243901	-0.66863935	1.003455	2046.888
## sigma_aS	1.81935950	0.263297486	1.437396759	2.27761625	1.001192	2904.589
## bS_dS	-0.21239629	0.011865632	-0.231437200	-0.19366649	1.000266	13050.617
## sigma_S	0.03429872	0.001806915	0.031506534	0.03728268	1.000996	11169.941

- This is very high eff and no divergent, as I can see in my synthetic data (I do not know yours,,). This is great. This model is reliably sampled to be used for analysis!

Posterior Check

```
post <- extract.samples(m2.3)
```

- Since I haven't touch since, I do not check the hidden parameters.

```
plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
  function(i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK,
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
```

```

lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

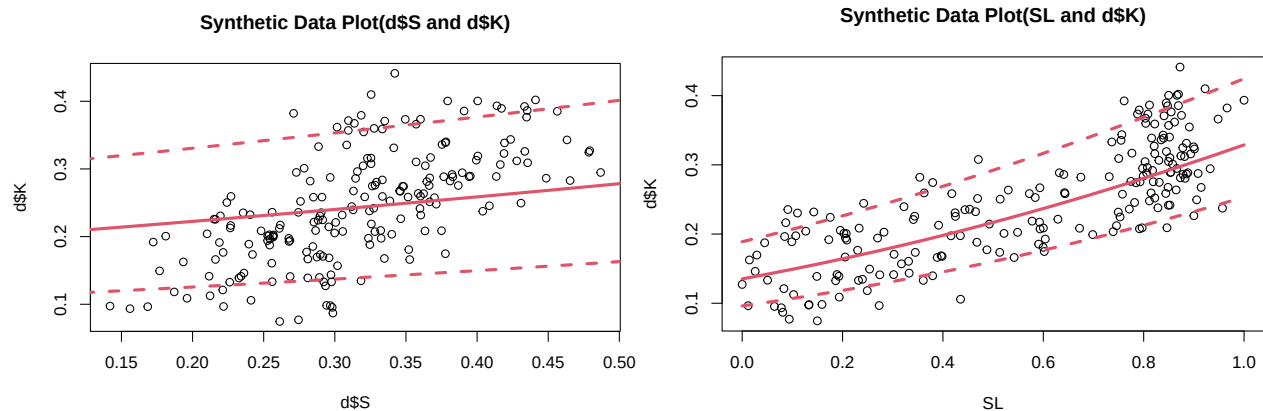
plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$S))
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

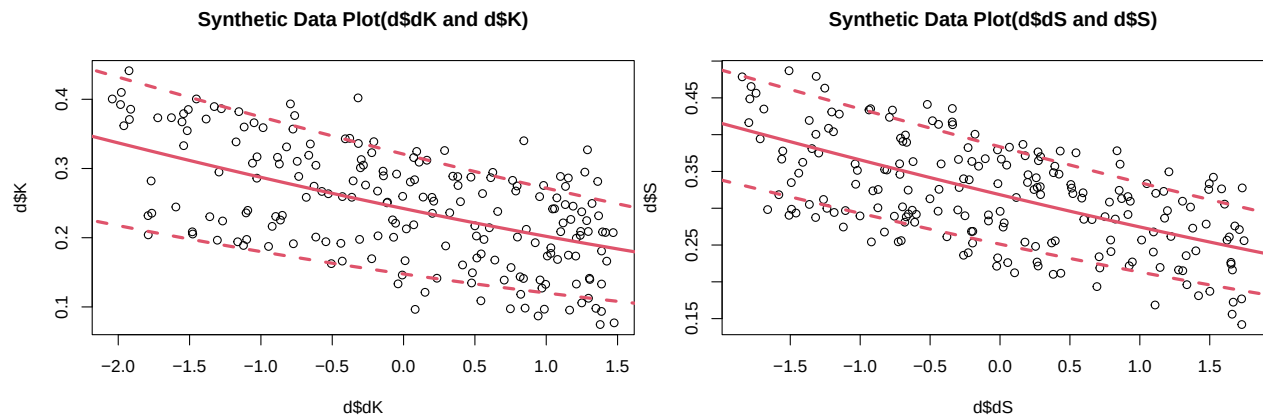
plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
  function (i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK_dK, 25) * d$dK))
means<- apply( KbydK , 2 , mean )
PIs<- apply( KbydK , 2 , PI )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S

plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```





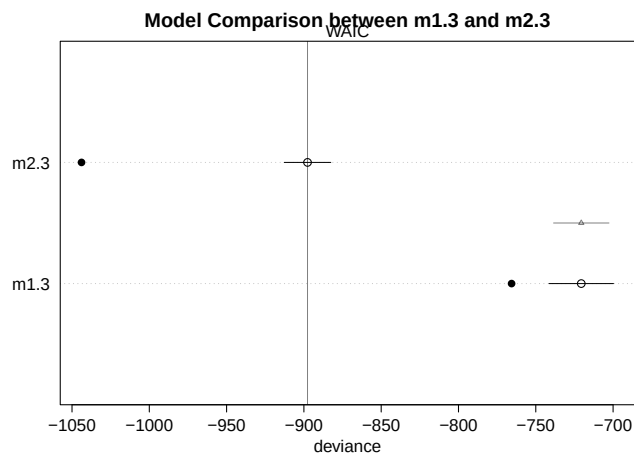
- The model 2.3 looks good. All posterior prediction plots shows inclusion of data points inside the model scope.

Compare Models

```
# Compare models
model_comparison <- compare(m1.3, m2.3)
print(model_comparison)

##           WAIC      SE  dWAIC    dSE  pWAIC      weight
## m2.3 -897.6593 15.06313  0.0000    NA 73.09431 1.000000e+00
## m1.3 -720.5791 20.86106 177.0802 17.9283 22.53785 3.527886e-39

# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m2.3")
```



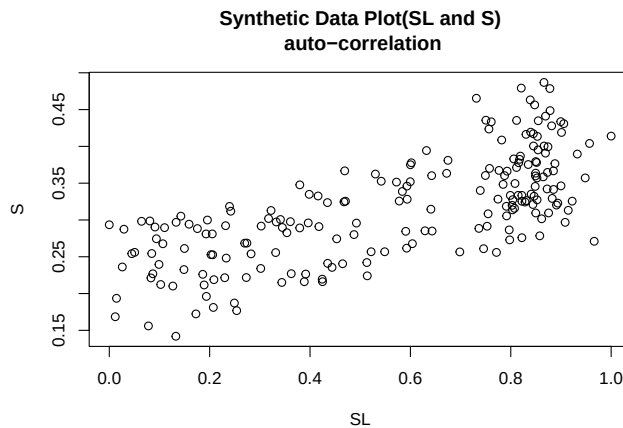
Key Columns in the compare Output:

- WAIC / PSIS: Lower values indicate better out-of-sample predictive accuracy.
- dWAIC / dPSIS: The difference in information criteria between the current model and the top-ranked (best) model.
- dSE: If the standard error is large relative to the difference, the two models have statistically indistinguishable predictive capacities.
- weight: The Akaike weight, which represents the relative probability or evidence that a given model will make the best predictions among the set of compared models

=> Therefore, the m1 is better than the other one.

- but also I am wondering if I use correlation between S and SL instead of modeling Y in K model. This is the m2

```
plot(SL, S, main = "Synthetic Data Plot(SL and S)\n auto-correlation")
```



This is because there is a correlation!!

Model 3 (m3)

- see priors in overview.pdf, but this include correlation between SL and S.
- Instead, this does not include multilevel model of $Y \rightarrow S$.
- This is based on the Ch 14.3 in Statistical Rethinking as SL and S are not centred
- warnings are annoying but relatively few so I just omit it for m2.2 and m3.

```
# Model 3
m3<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_Y[T] * Y[i] + bK_dK * dK,

    # In this case (m3), we do include correlation between S and SL due to Y.
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    vector [25]: bK_Y <- bK_Y_bar + sigma_bK_Y * z_bK_Y[T],
    c(aK_bar, bK_Y_bar)~normal(0,0.3),
    c(sigma_aK, sigma_bK_Y) ~ exponential(6),
    z_aK[T]~normal(0,0.1),
    z_bK_Y[T] ~ normal(0,0.1),

    transpars>vector[25]:bK_S <- v[,1],
    transpars>vector[25]:bK_SL <- v[,2],

    transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
    vector[2]: sigmaSSL ~ exponential(2),
    cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1), # I do not expect very very strong corre
    matrix[2,25]: zSSL ~ normal(0 , 1),

    # don't need regularization
    bK_dK ~ normal(0, 0.3),
    U ~ exponential(1),

    # Model S
    # This is correlation so I do not include multi-model of Y-> S
```

```

S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_dS * dS ,

vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
aS_bar ~ normal(0,0.3),
sigma_aS ~ exponential(6),
z_aS[T] ~ normal(0,0.1),

bS_dS ~ normal(0,0.3),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T, This should be hard coded based on the previous research
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treedepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```

## In file included from stan/src/stan/model/model_header.hpp:5:
## stan/src/stan/model/model_base_crtp.hpp:136:8: warning: 'void stan::model::model_base_crtp<M>::write_array(
## 136 | void write_array(stan::rng_t& rng, Eigen::VectorXd& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f3049073150.hpp:1606:3: note: by 'void ulam_cmdstanr::
## 1606 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:205:8: warning: 'void stan::model::model_base_crtp<M>::write_array(
## 205 | void write_array(stan::rng_t& rng, std::vector<double>& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f3049073150.hpp:1606:3: note: by 'void ulam_cmdstanr::
## 1606 | write_array(RNG& base_rng, Eigen::Matrix<double,-1,1>& params_r,
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:154:17: warning: 'double stan::model::model_base_crtp<M>::log_prob(
## 154 | inline double log_prob(std::vector<double>& theta, std::vector<int>& theta_i,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f3049073150.hpp:1645:3: note: by
## 'T_ ulam_cmdstanr_a684a685757990b7a510e9ec4271b65b_model_namespace::ulam_cmdstanr_a684a685757990b7a510e9ec4271b65b_model_namespace::
## 1645 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:91:17: warning: 'double stan::model::model_base_crtp<M>::log_prob(
## 91 | inline double log_prob(Eigen::VectorXd& theta,
##      | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f3049073150.hpp:1645:3: note: by 'T_ ulam_cmdstanr::
## 1645 | log_prob(Eigen::Matrix<T,-1,1>& params_r, std::ostream* pstream = nullptr) const {
##      | ~~~~~

```

```

## stan/src/stan/model/model_base_crtp.hpp:159:20: warning: 'stan::math::var stan::model::model_base_crtp::
## 159 | inline math::var log_prob(std::vector<math::var>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f3049073150.hpp:1645:3: note: by 'T_ulam_cmdst
## T_a, -1, 1>&, std::ostream*) const'
## 1645 | log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~
## stan/src/stan/model/model_base_crtp.hpp:96:20: warning: 'stan::math::var stan::model::model_base_crtp::
## 96 | inline math::var log_prob(Eigen::Matrix<math::var, -1, 1>& theta,
## | ~~~~~
## C:/Users/nobut/AppData/Local/Temp/RtmpM7wHH9/model-f3049073150.hpp:1645:3: note: by 'T_ulam_cmdst
## 1645 | log_prob(Eigen::Matrix<T_,-1,1>& params_r, std::ostream* pstream = nullptr) const {
## | ~~~~~

## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
##
## Chain 1 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 1
## Chain 2 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 2 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 2 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Us
## Chain 2 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 2 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 2
## Chain 3 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in 'C:/Users/nobut/AppD
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 3
## Chain 4 Iteration: 1 / 4000 [ 0%] (Warmup)
## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Us
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covar
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or m
## Chain 4
## Chain 4 Iteration: 100 / 4000 [ 2%] (Warmup)

```



```

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: lkj_corr_cholesky_lpdf: Random variable[2] is 0, but must be positive! (in 'C:/Users/
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 4

## Chain 2 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration: 100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 1 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration: 200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [10%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [10%] (Warmup)
## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [10%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [10%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [12%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [12%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [12%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [15%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [12%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [15%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [15%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [17%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [15%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [20%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [17%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [17%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [20%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [22%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [22%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [22%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [25%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [27%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [30%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [22%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [27%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [32%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [25%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [30%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [30%] (Warmup)

```

```

## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)

```

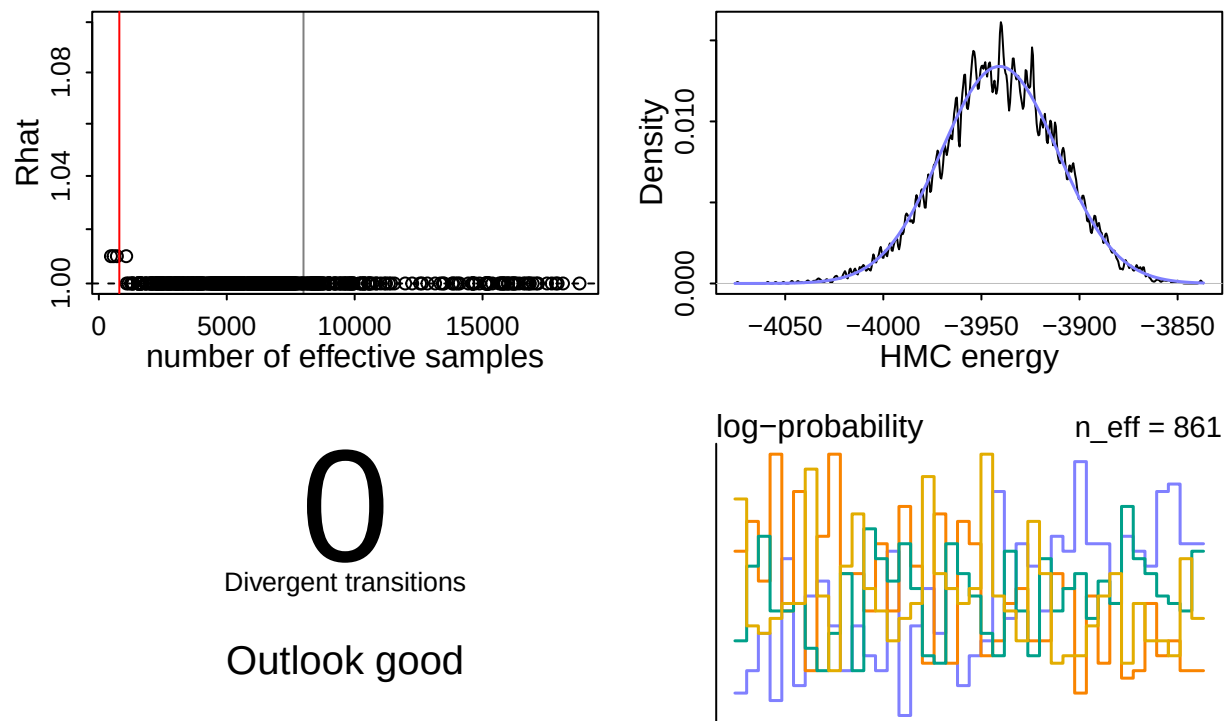
```

## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 126.7 seconds.
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 156.3 seconds.

```

```
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 158.9 seconds.
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 211.1 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 163.3 seconds.
## Total execution time: 211.3 seconds.
```

```
dashboard(m3)
```



-> Looks bad sampling efficiency.

Therefore, I will tackle this problem with the same approach though I do not want to remove the correlation parameters. Let me remove one more thing so that it corresponds m1.3.

```
# Model 3.2
m3.2<- ulam(
  alist (
    # Main model K
    K ~ normal(mu_K, U),
    logit(mu_K) <- aK[T] + bK_S[T] * S + bK_SL[T] * SL[i] + bK_dK * dK,
    vector[25]: aK <- aK_bar + sigma_aK * z_aK[T],
    aK_bar~normal(0,0.3),
```

```

sigma_aK ~ exponential(6),
z_aK[T] ~ normal(0,0.1),
transpars>vector[25]:bK_S <- v[,1],
transpars>vector[25]:bK_SL <- v[,2],
transpars>matrix[25, 2]:v <- compose_noncentered(sigmaSSL, RhoSSL, zSSL),
vector[2]: sigmaSSL ~ exponential(2),
cholesky_factor_corr[2]:RhoSSL ~ lkj_corr_cholesky(1),
matrix[2,25]: zSSL ~ normal(0 , 1),
bK_dK ~ normal(0, 0.3),
U ~ exponential(1),

# Model S
S ~ normal(mu_S, sigma_S),
logit(mu_S) <- aS[T] + bS_dS * dS ,
vector[25]: aS<-aS_bar + sigma_aS * z_aS[T],
aS_bar ~ normal(0,0.3),
sigma_aS ~ exponential(6),
z_aS[T] ~ normal(0,0.1),
bS_dS ~ normal(0,0.1),
sigma_S ~ exponential(1),

# Model SL by Y
vector[200]: SL ~ normal(inv_logit(6 * Y - 2) - 0.04, 0.05),

# Model T
T ~ poisson(Y * 25),

# Model T by Y
vector[200]:Y ~ normal( 0.5, 0.2)
), dat = d, constraints = list(
  SL = "lower=0,upper=1",
  Y = "lower=0,upper=1"
), chains=4, cores=4, iter= 4000, warmup = 2000, log_lik=TRUE, max_treepdepth = 13, control = list(
  adapt_delta = 0.99
)
)

```

```
## Running MCMC with 4 parallel chains, with 1 thread(s) per chain...
```

```
##
## Chain 1 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 2 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 4 Iteration:    1 / 4000 [ 0%] (Warmup)
## Chain 3 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 2 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 4 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 1 Iteration:  100 / 4000 [ 2%] (Warmup)
## Chain 3 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 2 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 4 Iteration:  200 / 4000 [ 5%] (Warmup)
## Chain 3 Iteration:  300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration:  200 / 4000 [ 5%] (Warmup)

```

```

## Chain 2 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 3 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 1 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 2 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 300 / 4000 [ 7%] (Warmup)
## Chain 1 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 4 Iteration: 400 / 4000 [ 10%] (Warmup)
## Chain 2 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 3 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 2 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 4 Iteration: 500 / 4000 [ 12%] (Warmup)
## Chain 1 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 2 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 600 / 4000 [ 15%] (Warmup)
## Chain 3 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 2 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 1 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 4 Iteration: 700 / 4000 [ 17%] (Warmup)
## Chain 3 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 4 Iteration: 800 / 4000 [ 20%] (Warmup)
## Chain 2 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 3 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 2 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 4 Iteration: 900 / 4000 [ 22%] (Warmup)
## Chain 1 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 1 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 2 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 4 Iteration: 1000 / 4000 [ 25%] (Warmup)
## Chain 3 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 2 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 4 Iteration: 1100 / 4000 [ 27%] (Warmup)
## Chain 3 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 2 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1200 / 4000 [ 30%] (Warmup)
## Chain 1 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 3 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 4 Iteration: 1300 / 4000 [ 32%] (Warmup)
## Chain 1 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 2 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 4 Iteration: 1400 / 4000 [ 35%] (Warmup)
## Chain 1 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 2 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 2 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1600 / 4000 [ 40%] (Warmup)

```

```

## Chain 4 Iteration: 1500 / 4000 [ 37%] (Warmup)
## Chain 3 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 2 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 4 Iteration: 1600 / 4000 [ 40%] (Warmup)
## Chain 3 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 2 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 4 Iteration: 1700 / 4000 [ 42%] (Warmup)
## Chain 1 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 3 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 2 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 2 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 4 Iteration: 1800 / 4000 [ 45%] (Warmup)
## Chain 1 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 3 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 3 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 2 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 1 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 1900 / 4000 [ 47%] (Warmup)
## Chain 1 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 2 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 1 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 4 Iteration: 2000 / 4000 [ 50%] (Warmup)
## Chain 4 Iteration: 2001 / 4000 [ 50%] (Sampling)
## Chain 3 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 2 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 1 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 4 Iteration: 2100 / 4000 [ 52%] (Sampling)
## Chain 3 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 2 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 1 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 4 Iteration: 2200 / 4000 [ 55%] (Sampling)
## Chain 3 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 2 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 4 Iteration: 2300 / 4000 [ 57%] (Sampling)
## Chain 3 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 2 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 4 Iteration: 2400 / 4000 [ 60%] (Sampling)
## Chain 3 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 2 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 4 Iteration: 2500 / 4000 [ 62%] (Sampling)
## Chain 1 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 3 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 2 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 4 Iteration: 2600 / 4000 [ 65%] (Sampling)
## Chain 1 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 3 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 2 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2700 / 4000 [ 67%] (Sampling)
## Chain 1 Iteration: 2900 / 4000 [ 72%] (Sampling)

```

```

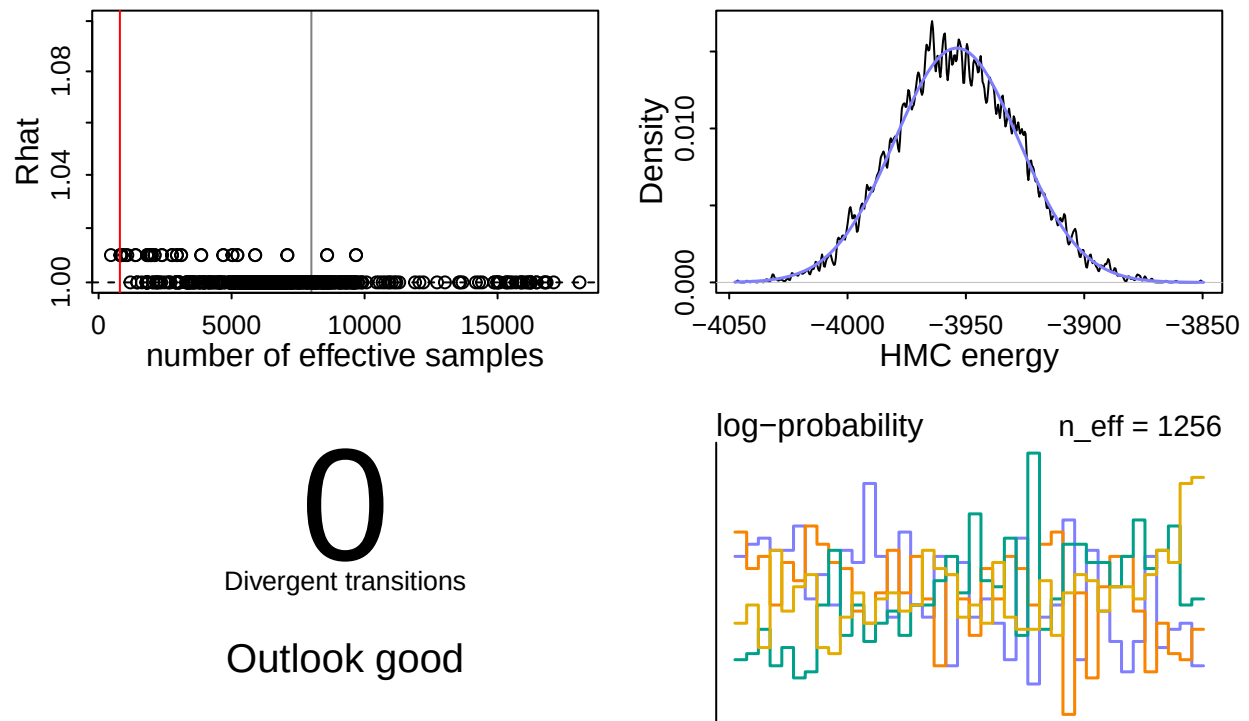
## Chain 2 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 3 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 4 Iteration: 2800 / 4000 [ 70%] (Sampling)
## Chain 1 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 3 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 4 Iteration: 2900 / 4000 [ 72%] (Sampling)
## Chain 1 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 3 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 1 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 4 Iteration: 3000 / 4000 [ 75%] (Sampling)
## Chain 2 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 3 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 1 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 4 Iteration: 3100 / 4000 [ 77%] (Sampling)
## Chain 2 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 3 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 1 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 4 Iteration: 3200 / 4000 [ 80%] (Sampling)
## Chain 2 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 3 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 1 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 4 Iteration: 3300 / 4000 [ 82%] (Sampling)
## Chain 2 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 3 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 1 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 4 Iteration: 3400 / 4000 [ 85%] (Sampling)
## Chain 2 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 3 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 1 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 4 Iteration: 3500 / 4000 [ 87%] (Sampling)
## Chain 2 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 1 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 4 Iteration: 3600 / 4000 [ 90%] (Sampling)
## Chain 2 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 3 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 1 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 3700 / 4000 [ 92%] (Sampling)
## Chain 2 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 2 finished in 77.0 seconds.
## Chain 3 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 1 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 1 finished in 77.4 seconds.
## Chain 4 Iteration: 3800 / 4000 [ 95%] (Sampling)
## Chain 3 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 3 finished in 78.9 seconds.
## Chain 4 Iteration: 3900 / 4000 [ 97%] (Sampling)
## Chain 4 Iteration: 4000 / 4000 [100%] (Sampling)
## Chain 4 finished in 81.3 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 78.6 seconds.

```



```
## Total execution time: 81.5 seconds.
```

```
dashboard(m3.2)
```



```
precis(m3.2, depth=1)
```

##		mean	sd	5.5%	94.5%	rhat	ess_bulk
##	aK_bar	-1.35827874	0.063072339	-1.46042738	-1.25817295	1.0003124	4134.128
##	sigma_aK	0.18875223	0.195159911	0.01074130	0.57223605	1.0008199	6305.297
##	bK_dK	-0.24220021	0.012532506	-0.26218367	-0.22210123	1.0006117	13553.612
##	U	0.03024720	0.001955600	0.02705753	0.03329903	1.0047827	1195.163
##	aS_bar	-0.74415295	0.049983505	-0.82308945	-0.66281037	1.0010925	2298.520
##	sigma_aS	1.81911049	0.266099093	1.43576574	2.27470237	0.9999758	4402.188
##	bS_dS	-0.20964045	0.011962093	-0.22874676	-0.19026328	1.0012463	14183.694
##	sigma_S	0.03432137	0.001822906	0.03155078	0.03737507	1.0003040	11022.176

- This model is good enough.

Posterior Check

```
post <- extract.samples(m3.2)
```

- I do not check the Hidden parameters since I have never touched it since m1

Correlation?

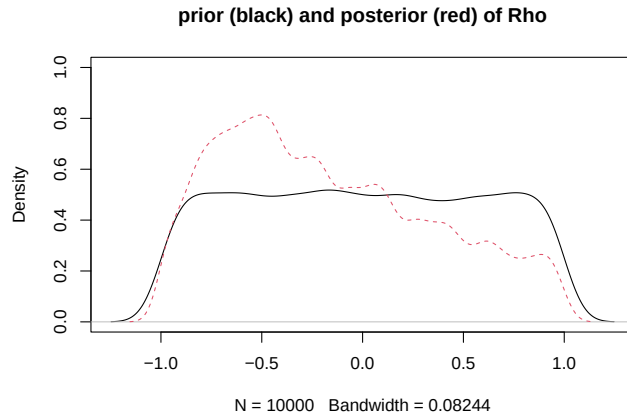
- Rho value shows a bit of correlation between S and SL.

```
R <- rljcorr( 1e4 , K=2 , eta=1 )# eta min lkj_corr()
plot(density( R[,1,2]), ylim = c(0, 1), main = "prior (black) and posterior (red) of Rho")
# This is complicated as cholesky correlation
true_rho_samples=NULL
for (i in 1:2000) {
```

```

L <- post$RhoSSL[i, , ]
R <- L %*% t(L)
true_rho_samples[i] <- R[1, 2]
}
dens(true_rho_samples, add = TRUE, lty = 2, col = 2)

```



Posterior Predictions

```

plot(d$S, d$K, main = "Synthetic Data Plot(d$S and d$K)")
Sseq<-seq(0, 0.5, len = 20)
KbyS<- sapply(Sseq, # Tricky but t and t to calculate estimates by group, though dK does not have group
function(i) inv_logit(post$aK + post$bK_S * i + t(t(post$bK_SL) * colMeans(post$SL)) + rep(post$bK,
means<- apply( KbyS , 2 , mean )
PIs<- apply( KbyS , 2 , PI )
lines(Sseq, means, col = 2, lwd = 3)
lines(Sseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(Sseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(SL, d$K, main = "Synthetic Data Plot(SL and d$K)")
SLseq<-seq(0, 1, len = 20)
KbySL<- sapply(SLseq,
function(i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + post$bK_SL * i + rep(post$bK_dK, 25) * d$S)
means<- apply( KbySL , 2 , mean )
PIs<- apply( KbySL , 2 , PI )
lines(SLseq, means, col = 2, lwd = 3)
lines(SLseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(SLseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

plot(d$dK, d$K, main = "Synthetic Data Plot(d$dK and d$K)")
dKseq<-seq(-2.5, 2, len = 20)
KbydK<- sapply(dKseq,
function(i) inv_logit(post$aK + t(t(post$bK_S) * d$S) + t(t(post$bK_SL) * colMeans(post$SL)) + rep
PIs<- apply( KbydK , 2 , PI )
means<- apply( KbydK , 2 , mean )
lines(dKseq, means, col = 2, lwd = 3)
lines(dKseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dKseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

# Model S
plot(d$dS, d$S, main = "Synthetic Data Plot(d$dS and d$S)")

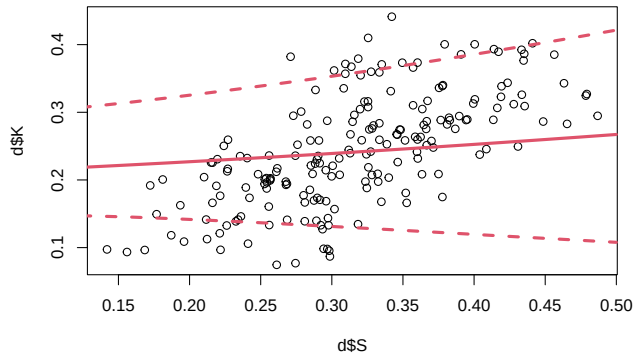
```

```

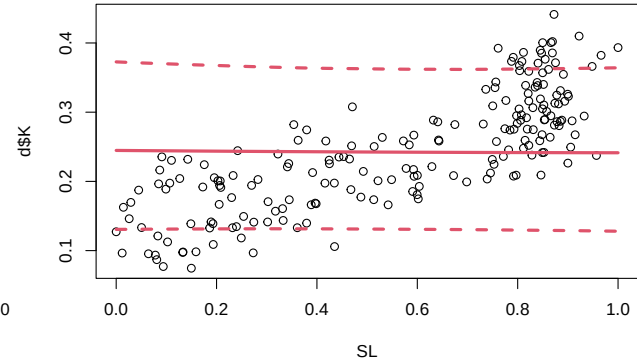
dSseq<-seq(-2, 2, len = 20)
SbydS<- sapply(dSseq,
  function (i) inv_logit(post$aS + rep(post$bS_dS, 25) * i )) # Used mean of each simulated data
PIs<- apply( SbydS , 2 , PI )
means<- apply( SbydS , 2 , mean )
lines(dSseq, means, col = 2, lwd = 3)
lines(dSseq, PIs[1, ], col = 2, lty = 2, lwd = 3)
lines(dSseq, PIs[2, ], col = 2, lty = 2, lwd = 3)

```

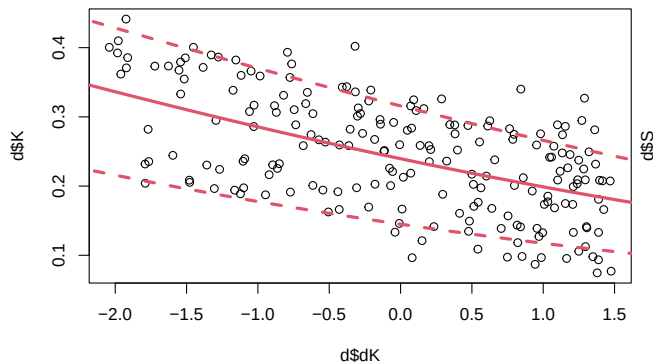
Synthetic Data Plot(d\$S and d\$K)



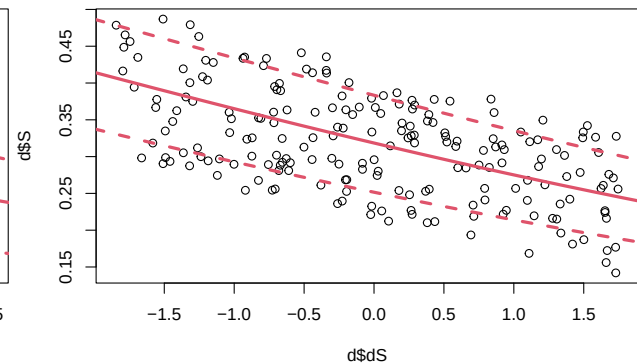
Synthetic Data Plot(SL and d\$K)



Synthetic Data Plot(d\$dK and d\$K)



Synthetic Data Plot(d\$dS and d\$S)



Overall K estimation is fine.

let me compare all those models.

Compare Models

```

# Compare models
model_comparison <- compare(m1.3, m3.2)
print(model_comparison)

##           WAIC          SE    dWAIC      dSE    pWAIC      weight
## m3.2 -797.4967 19.90267  0.00000      NA 32.15345 1.00000e+00
## m1.3 -720.5791 20.86106 76.91764 18.61264 22.53785 1.98402e-17

# Plot WAIC/PSIS differences
plot(model_comparison, main = "Model Comparison between m1.3 and m3.3")

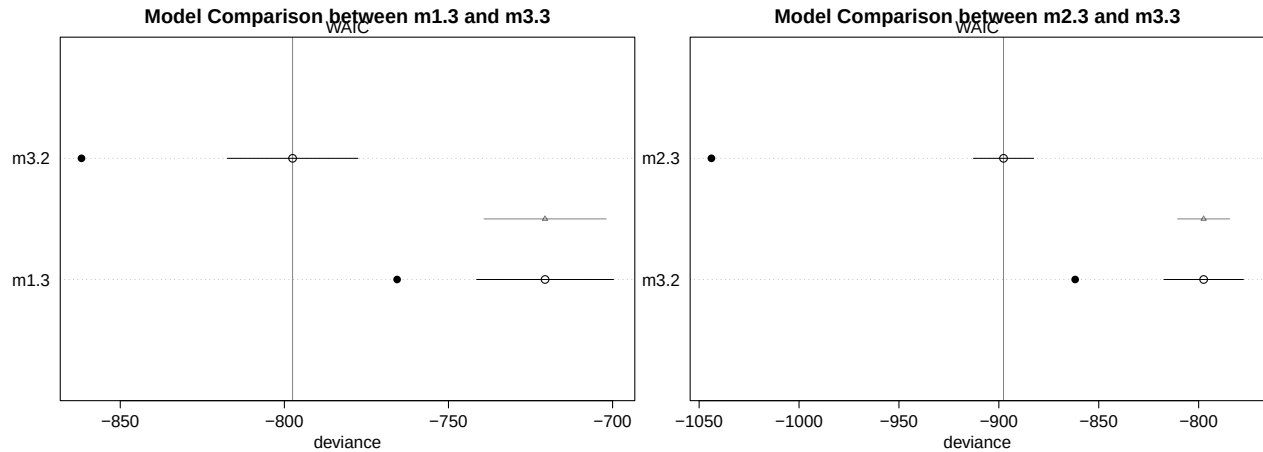
model_comparison <- compare(m2.3, m3.2)
print(model_comparison)

```

```
##           WAIC           SE    dWAIC      dSE    pWAIC      weight
## m2.3 -897.6593 15.06313    0.0000      NA 73.09431 1.000000e+00
## m3.2 -797.4967 19.90267 100.1626 13.05229 32.15345 1.778151e-22
```

```
# Plot WAIC/PSIS differences
```

```
plot(model_comparison, main = "Model Comparison between m2.3 and m3.3")
```



This looks Model 2 is a bit better in this case.

Future Suggestions

- Removing causal relationship in each model (for example, no causal relationship from Y to K) may be examined with actual data.
- The Model 3 still has a problem of bK_T and bS_T as they do not fit the data. I am trying to understand the reason and fix it.

```
rm(list=ls())
```